SCOPE OF WORK

AI-Assisted Development Workflow Implementation

Claude Code Enablement — Foundation, Knowledge-Base Generation & Rollout

Client	Ibaset
Prepared by	Brilworks Technology Pvt Ltd (www.brilworks.com)
Contact	Vikas Singh, CTO & Co-founder · vikas@brilworks.com · +91-8866518364
Service	AI Development Intelligence — Implementation & Enablement
Date	May 2026
Version	1.0
Timeline	Phase 1: ~3–4 weeks · Support: ~1 quarter · Phase 2: after LOC audit
Investment	Phase 1: USD 11,000 fixed (incl. IBA microservice ~300K LOC + 40 hrs support) · Phase 2: per-repo after audit · Optional support: USD 3,500/quarter
Confidentiality	Confidential

503, Fortune Business Hub, Science City Road, Sola, Ahmedabad, Gujarat, India 380060
sales@brilworks.com • +91 9313644148

Table of Contents

1. Executive Summary

Ibaset is adopting an AI-assisted development workflow on its own codebase, built around the Claude Code CLI and three commands the team runs every day: **/plan** (build the implementation plan and a shared API contract), **/review-plan** (review the plan and the code against it, looping back to improve the plan), and **/implement-ticket** (write code from the approved plan and run a pre-QA self-test).

This is an implementation and enablement service, not a software licence. The engine of the workflow is the knowledge base — Markdown files, generated from the codebase, database and existing documentation, that let the commands understand this specific product. Generating the knowledge base across a large repository estate is the major, time-consuming effort, so the engagement is structured to de-risk it: prove the approach on a small set of reference repositories first, measure the estate, then generate at scale using whichever delivery model the client prefers.

Because the product knowledge sits with the client, the accuracy of every generated knowledge file is verified by the client in all delivery models — Brilworks guarantees the structure and quality of the files, not the domain facts inside them.

Note — start here

Generating the knowledge base is the most important early step in this engagement. The commands are only as good as the knowledge they read, so the work starts by generating and proving that knowledge base on reference repositories before anything is scaled. It is also the most time-consuming part — which is why it is measured (the LOC audit) before the estate-wide cost is committed.

Engagement at a glance

- Phase 1 — Foundation & Reference Model: stand up the three commands, the MCP integrations and the rules framework; generate the knowledge base for the IBA microservice (~300,000 LOC, known) — from the code, the database schema and existing Confluence docs — as the reference model; and run a lines-of-code (LOC) audit across the rest of the estate to size Phase 2.
- Phase 2 — Knowledge-Base Generation at Scale: generate the knowledge base across the in-scope repositories, priced after the audit, with two delivery options (Brilworks-led or hybrid client-led with our templates and QA).
- Optimization & Support — included in Phase 1: 40 hours over ~1 quarter of weekly /review-plan feedback and tuning of the commands, rules and knowledge (first 10 hours free). An optional continued-improvement package — 120 hours per quarter for USD 3,500 — is available thereafter.
- Phase 3 — Future Knowledge Source (not estimated here): TestRail as an additional input to the knowledge base.
- Stack: Java / Spring backend, Next.js / React (SSR) frontend, MySQL. Estate: ~21 internal + ~18 external repositories (LOC to be measured).
- Integrations: Jira, Figma, Slack, Sentry, local code-search and a local image / audio / video MCP (MySQL optional). Phase 1 investment: USD 11,000 (fixed; includes the IBA microservice

> reference build and 40 hours of support).

2. Background — The Problem We Are Solving

On a typical sprint day, a developer starts coding before the approach is fully thought through, then discovers mid-flight that the change touches more of the system than expected, breaks an interface the frontend relied on, or violates a convention caught only at review. The result is repeated QA cycles and avoidable rework. The root causes are consistent across teams:

- No systematic check, before coding, of which parts of the codebase a change affects.
- No shared contract between backend and frontend engineers, so integration breaks late.
- Plans and conventions live in people's heads, so quality depends on who picks up the ticket.
- Compliance and convention issues are caught in PR review, not during development.

The workflow addresses each directly: **/plan** produces a concrete plan with affected files and a shared API contract before coding; **/review-plan** validates the plan and the implementation against it, feeding corrections back so the plan command keeps improving; and **/implement-ticket** writes code that follows the team's patterns and runs a pre-QA self-test that removes most rework.

3. Solution Overview

3.1 The three commands

Command	When it's used	What it produces
/plan	A ticket is ready to be planned (auto-detects backend / frontend scope)	Affected files, a shared API contract, ordered implementation steps, a compliance checklist and P0/P1 test scenarios
/review-plan	A plan or an implementation needs checking — sits between /plan and /implement	A review of the plan (and code vs. plan); gaps and corrections are fed back to improve the plan. Use it to loop back before implementing
/implement-ticket	A plan has been reviewed and approved	Working code on a branch, with a pre-QA self-test and a compliance scan

Two design points carry the value. First, the API contract written by /plan lets backend and frontend engineers implement in parallel against the same shapes — removing the integration surprises that normally appear at merge. Second, /review-plan forms a learning loop: its corrections improve the planning context over time, so plans get sharper the more the team uses the workflow.

3.2 The four pillars Brilworks builds

Pillar	What it is	Ownership
Knowledge files	Markdown files describing the codebase — pattern files and per-domain files — generated from the backend/API code, the frontend, the SQL database schema and existing Confluence documentation.	Brilworks generates (Phase 1 model repos); pattern files reviewed and standardised by the client. See Section 4.
MCP servers	Tools the commands call automatically: Jira, Figma, Slack, Sentry, local code-search and a local image / audio / video MCP (MySQL optional).	Brilworks configures; client supplies access and API tokens.
Coding rules	Auto-loaded standards in .claude/rules/ (security, API conventions, frontend conventions, testing).	Client provides current BE/FE standards; Brilworks supplies sample rule files and integrates the standards; client generates the rest from our samples.
Claude Code CLI	Installed per developer machine; reads .mcp.json; runs the commands; setup.sh syncs config.	Brilworks sets up; runs on each developer's machine.

4. The Knowledge Base — The Core Asset

The commands are generic; what makes them accurate for this product is the knowledge base. It is the single most important — and most time-consuming — deliverable, which is why the engagement is built around generating it safely and at scale.

4.1 What it is built from

Source	Produces	When
Backend / API codebase	Controller, service and entity conventions; per-domain knowledge files	Phase 1 & 2
Frontend codebase	Next.js / React (SSR) patterns, data fetching and state conventions	Phase 1 & 2
SQL database (MySQL)	Schema references	Phase 1 & 2
Confluence	Knowledge generated from existing documentation pages	Phase 1 & 2
TestRail	Knowledge generated from existing test cases	Phase 3 (future)

4.2 How it is generated — methodology & throughput

Knowledge files are generated by running Claude sessions over the source, then reviewed and enriched with business context. Based on Brilworks' experience, the generation rate is predictable:

Generation throughput (observed)

- ≈ 500 lines of code → ≈ 1 Claude session → ≈ 1 minute to generate the knowledge file for it.
- 5–7 such sessions can run in parallel on a single machine, so wall-clock time scales with available machines.
- Generation is fast; the gating effort is human review and enrichment, and the client's standardisation of the pattern files.

Because generation effort tracks the total lines of code, **Phase 1 includes a LOC audit across all ~39 repositories** to convert the estate into a concrete time-and-cost estimate for Phase 2. No full-estate generation is priced until that measurement exists.

4.3 Two delivery models for generation at scale

Option A — Brilworks-led: Brilworks generates, reviews and QAs the knowledge base for all in-scope repositories. Lowest client effort, highest cost.

Option B — Hybrid (client-led with our model): Brilworks builds the reference model and templates and trains the team; the client team generates the bulk of the repositories following our sample, and Brilworks reviews and QAs the output and spot-enriches. Lower cost, shares the load with the client team.

Pattern files are reviewed and standardised by the client team in both options, so the generated knowledge reflects the conventions the client wants enforced.

4.4 Verification & ownership of accuracy

Brilworks builds and QAs the structure of the knowledge base, but does not hold the client's product or domain expertise. The accuracy of the product information inside the knowledge files is therefore verified by the client — in both delivery models.

Who verifies what

- Brilworks (both options): the structure and quality of the generated knowledge files — correct sections, file paths, references and format.
- Client (both options): the product/domain accuracy of the generated knowledge — whether the information is correct and complete for the product.
- In particular, the client verifies the high-level files that the commands rely on most: each repository's INDEX.md and the CLAUDE.md.

5. Scope of Work

5.1 Phase 1 — Foundation & Reference Model

Goal: a working end-to-end workflow on the IBA microservice (the Phase 1 reference repository, ~300,000 LOC), plus everything needed to generate the rest at scale and a measured estimate for doing so.

Brilworks will:

1. Run discovery across the repository estate; the IBA microservice (~300,000 LOC, known) is the designated Phase 1 reference repository (one further small repository may be added if useful).
2. Stand up the three commands (/plan, /review-plan, /implement-ticket), the CLI and setup.sh on the reference team's machines.
3. Configure the MCP servers: Atlassian / Jira, Figma, Slack, Sentry, local code-search and a local image / audio / video MCP (MySQL optional).
4. Provide sample pattern files (api-patterns, frontend-patterns, database-patterns, testing-patterns) and sample rule files (.claude/rules/), and integrate the client's current backend and frontend coding standards.
5. Generate and structurally QA the knowledge base for the IBA microservice (~300,000 LOC) — pattern files, per-domain files, each INDEX.md, the CLAUDE.md and DB schema references — as the working reference model.
6. Integrate existing Confluence documentation as a knowledge-base source for the reference scope, and document the approach for the wider estate.
7. Run a LOC audit across all ~39 repositories and produce a sizing model and a Phase 2 cost estimate for both delivery options.
8. Document the generation methodology and a QA checklist — the runbook for parallelising 5–7 sessions per machine and validating output.
9. Demonstrate the workflow with the /plan command on ~10 tickets, targeting ~70% plan accuracy at this stage. Real-ticket end-to-end testing may be limited in Phase 1 because it covers a single microservice; accuracy improves through the /review-plan feedback loop and the optimization support.
10. Train the client team on generating the knowledge base, so the hybrid option is viable, and deliver a baseline + readout.

5.2 Phase 2 — Knowledge-Base Generation at Scale

Goal: the knowledge base generated across the in-scope repositories so the workflow works estate-wide. Scope and price are confirmed from the Phase 1 LOC audit; the client selects Option A or Option B.

- **Option A — Brilworks-led:** Brilworks generates, reviews, enriches and QAs the knowledge base for each repository, repo by repo.
- **Option B — Hybrid:** the client team generates each repository from our templates and training; Brilworks reviews and QAs the output, spot-enriches, and integrates it into the commands.
- Both options: client review and standardisation of pattern files; integration of the rules the client has generated from our samples.
- Both options: the client verifies the product/domain accuracy of the generated knowledge files (including each repository's INDEX.md and the CLAUDE.md); Brilworks QA covers structure and quality.

5.3 Ongoing Optimization & Support (Recommended — One Quarter)

Embedding the workflow takes time. As the team uses /plan, the output they review may not always be relevant or 100% accurate, so they use /review-plan and feed corrections back to the AI. Each week the client team shares this feedback with Brilworks, who review it and optimise the commands, rules and knowledge files.

- The team runs /plan, reviews the output with /review-plan, and feeds corrections back.
- Each week the client shares the feedback with Brilworks; Brilworks reviews it and optimises the commands, rules and knowledge files.
- Brilworks includes the first 10 hours of this support free.
- Recommended duration: at least one quarter (~3 months) — embedding the workflow into the team's habits takes that long. 40 hours of this support are included in the Phase 1 price (see Section 9.1); the first 10 hours are offered free.
- Beyond the 40 hours included in Phase 1, an optional quarterly continued-improvement package is available (120 hours per quarter for USD 3,500; additional hours at USD 30/hr — see Section 10.1).

5.4 Phase 3 — Future Knowledge Source (not estimated here)

- TestRail — generate knowledge base content from existing test cases.

This is noted for roadmap context and will be scoped and estimated as a separate addendum when prioritised. (Confluence is included from Phase 1.)

6. Scope Split — Who Does What

Brilworks delivers

- The three commands (/plan, /review-plan, /implement-ticket), the CLI and setup.sh.
- MCP configuration: Jira, Figma, Slack, Sentry, local code-search and a local image / audio / video MCP (MySQL optional).
- Sample pattern files and sample rule files; integration of the client's coding standards.
- Reference-repo knowledge base (code, schema and Confluence), the LOC audit and the Phase 2 sizing model.
- Generation methodology / QA runbook, team training, and structural QA of generated knowledge. (Option A: full generation at scale.)
- Weekly review of the team's /review-plan feedback and optimisation of commands, rules and knowledge (first 10 hours free).

Client owns

- Current backend and frontend coding standards.
- Review and standardisation of the pattern files; generating the rule files from our samples.
- Verifying the product/domain accuracy of all generated knowledge files in both options — including each repository's INDEX.md and the CLAUDE.md.
- (Option B) Generating the bulk of the knowledge base from our templates.

- Repository, database and tool access, API tokens and Anthropic capacity.
- Third-party and Anthropic usage costs, including knowledge-generation compute.

Shared

- Selecting the reference repositories.
- Agreeing the conventions encoded in the rules.
- Agreeing the QA acceptance criteria for generated knowledge.

7. Repository Inventory & Sizing

Generation effort tracks lines of code, so the estate must be measured before Phase 2 is priced. The Phase 1 audit produces per-repository LOC and converts it into a generation estimate.

Repository group	Count	Lines of code	Covered in
IBA microservice — Phase 1 reference repo	**1**	**~300,000 (known)**	**Phase 1 (included)**
Internal repositories (incl. IBA)	~21	To be measured (Phase 1 audit)	Phase 1 model + Phase 2
External repositories	~18	To be measured (Phase 1 audit)	Phase 2 (confirm inclusion after audit)
Total	**~39**	**TBD**	

The IBA microservice is one of the ~21 internal repositories, broken out because its size (~300,000 LOC) is already known; it is generated in Phase 1 as the reference model and included in the Phase 1 fixed price. At the observed rate (~500 LOC per session), it is roughly 600 generation sessions — a few hours of wall-clock time across parallel machines, plus structural QA. The remaining repositories are measured in the audit.

8. Timeline & Milestones

Milestone	Phase	Indicative timing	Output
Kickoff & discovery	1	≈1.5–2 wks*	Reference repo (IBA) confirmed; access confirmed; standards gathered
Workflow & integrations live	1	(combined)*	Three commands, MCP servers, sample rules/patterns, CLI on reference machines
Reference model + audit	1	≈1.5–2 wks*	IBA knowledge base (incl. Confluence); /plan demo on ~10 tickets (~70% accuracy); LOC audit + Phase 2 estimate; training & readout — Phase 1 acceptance
Generation at scale	2	After audit	Knowledge base generated across in-scope

Milestone	Phase	Indicative timing	Output
			repos (Option A or B), reviewed & integrated
Optimization & support (in Phase 1)	1	~3 months (1 quarter)	40 hrs included: weekly /review-plan feedback reviewed; commands, rules & knowledge tuned (first 10 hrs free)
Future source (TestRail)	3	When prioritised	TestRail — separate addendum

**Kickoff, discovery and workflow/integration setup together take ~1.5–2 weeks, subject to timely access to credentials and resources; the reference model (the IBA microservice) + audit then takes ~1.5–2 weeks (Phase 1 ≈ 3–4 weeks total). Phase 2 duration depends on the chosen option, repository count and the number of machines running generation in parallel, and is confirmed with the audit.*

9. Investment

Phase 1 is a fixed price (below). Phase 2 is a unit-priced workstream — per repository — confirmed after the Phase 1 LOC audit. All figures are proposed by Brilworks and open to adjustment.

9.1 Phase 1 — fixed price

Phase 1 — what's included
Discovery & reference-repo selection
Commands (/plan, /review-plan, /implement-ticket) + CLI + setup.sh
MCP configuration (Jira, Figma, Slack, Sentry, local code-search, media MCP; MySQL optional)
Sample pattern files + sample rule files + integration of client coding standards
Generate & QA the knowledge base for the IBA microservice (~300,000 LOC, known) — primary reference repository
Integrate Confluence documentation as a knowledge source (reference scope)
LOC audit across ~39 repositories + Phase 2 sizing model
Generation methodology + QA runbook
/plan demo on ~10 tickets (~70% accuracy at this stage) + tuning
Team training on knowledge-base generation
Baseline, readout & coordination
Optimization & support — 40 hours over ~1 quarter (first 10 hours free)

Phase 1 fixed price (everything above included)	**USD 11,000**

9.2 Phase 2 — generation pricing (confirmed after the audit)

Priced per repository, tiered by size. The client chooses one delivery model. Costs below assume the client pays Anthropic generation compute directly (see 9.3).

Option A — Brilworks-led

Repository size (LOC)	Brilworks effort / repo	Cost / repo (USD)
Small (< 25K)	~3 hrs	$150
Medium (25K – 100K)	~5 hrs	$250
Large (> 100K)	~9 hrs	$450

Option B — Hybrid (client-led with Brilworks templates + QA)

Repository size (LOC)	Brilworks QA / support / repo	Cost / repo (USD)
Small (< 25K)	~1 hr	$50
Medium (25K – 100K)	~1.5 hrs	$75
Large (> 100K)	~2.5 hrs	$125
One-time enablement & QA framework		**$1,200**

Illustrative total for 39 repos (assumes 20 small / 14 medium / 5 large — actuals from the audit)

- Option A (Brilworks-led): 20×$150 + 14×$250 + 5×$450 ≈ USD 8,750.
- Option B (Hybrid): $1,200 + 20×$50 + 14×$75 + 5×$125 ≈ USD 3,875.
- These are indicative only. The binding Phase 2 quote is issued from the Phase 1 LOC audit and the confirmed repository inclusion.

9.3 Client's own costs (paid directly, not to Brilworks)

Item	Notes	Indicative cost
Anthropic — daily workflow	Pay-as-you-go API tokens for /plan, /review-plan, /implement.	Low per ticket
Anthropic — knowledge generation	Generation compute scales with total LOC; sized in the Phase 1 audit.	Per LOC (audit)
Claude Code CLI	Free tool; requires an Anthropic API key or Claude subscription.	Per Anthropic plan
Jira / Figma / Slack / Sentry / Confluence	Existing client licences; API tokens required for the MCP servers.	No new cost
MCP server hosting	Runs locally / lightweight.	Negligible

Scope note on integrations
This SOW covers the integrations listed (Jira, Figma, Slack, Sentry, local code-search and a local image / audio / video MCP; MySQL optional). Any additional third-party integration must be collected and the estimate revised before it is included.

10. Optional & Future

10.1 Continued improvement support (optional)

Beyond the 40 hours included in Phase 1, an optional quarterly package keeps improving the workflow as the team uses it — reviewing /review-plan feedback and tuning the commands, rules and knowledge files.

Item	Detail	Cost (USD)
Quarterly support package	Up to 120 hours per quarter (~40 hrs / month) of continued improvement — tuning commands, rules and knowledge from /review-plan feedback	$3,500 / quarter
Additional hours	Beyond the 120 hours included in the quarter	$30 / hour

10.2 Other future options

Quoted for context only; not part of the committed scope. Each can be added as a separate addendum.

Option	What it covers
Phase 3 — TestRail source	Extend the knowledge base to generate from existing test cases (TestRail).
CI/CD integration	Run /review-plan in the pipeline and add PR gating so the self-test runs automatically.
Additional integrations / repos	New third-party MCP servers or repositories beyond the agreed estate.

11. Client Prerequisites

- Current backend and frontend coding standards (so they can be encoded into the rules).
- Read access to the repositories (reference repos for Phase 1; the wider estate for the audit and Phase 2).
- Access to the relevant Confluence space(s) for documentation-based knowledge generation.
- Access to a non-production MySQL database (or schema dump) for schema introspection — optional, only if the MySQL MCP is used.
- Access and API tokens for Jira, Figma, Slack and Sentry.

- A nominated technical point of contact and availability of the reference team for interviews and Q&A.
- Developer machines able to run the Claude Code CLI, and (for generation) machines able to run 5–7 parallel sessions.
- An Anthropic account / API key with sufficient capacity for the daily workflow and for knowledge-base generation.

12. Assumptions & Exclusions

12.1 Assumptions

- The estate is a Java / Spring backend, a Next.js / React (SSR) frontend, and a MySQL database.
- The IBA microservice (~300,000 LOC) is the Phase 1 reference repository; its size is treated as known and one further small repository may be added if useful.
- The ~500 LOC ≈ 1 session ≈ 1 minute generation rate holds approximately for this codebase; final Phase 2 sizing comes from the audit.
- The client provides current coding standards, access, credentials and timely availability.
- Brilworks does not hold the client's product/domain expertise; the accuracy of generated knowledge files is verified by the client in all delivery models, while Brilworks is responsible for their structure.
- Embedding the workflow takes time; an optimisation period of at least one quarter is recommended.
- Inclusion of the ~18 external repositories is confirmed after the audit.

12.2 Exclusions

- Building product features for the client — this is workflow enablement, not feature development.
- Provisioning or paying for third-party licences and Anthropic API/token costs (including generation compute).
- Integrations beyond those listed, until collected and re-estimated.
- TestRail source (Phase 3, separate addendum).
- Major refactoring of the existing codebase; ongoing maintenance after handoff (available as a retainer).

13. Acceptance Criteria

Phase 1 — acceptance

- The workflow is demonstrated on the IBA microservice: the /plan command is run on ~10 tickets, targeting ~70% plan accuracy at this stage (real-ticket end-to-end testing may be limited as Phase 1 covers a single microservice). /review-plan and /implement-ticket are demonstrated, and accuracy is expected to improve through the feedback loop.
- Reference-repo knowledge base (code, schema and Confluence), the MCP servers, sample

pattern files and sample rule files are in place and demonstrated.

- A LOC audit across the estate and a Phase 2 sizing/cost estimate (both options) are delivered, along with the generation methodology/runbook and team training.
- The client has verified the product/domain accuracy of the reference-repo knowledge files — including each INDEX.md and the CLAUDE.md — and Brilworks has confirmed their structure.

Phase 2 — acceptance

- Knowledge base generated and structurally QA-passed by Brilworks for the agreed in-scope repositories under the chosen option.
- The client has verified the product/domain accuracy of the generated knowledge files for those repositories (INDEX.md and CLAUDE.md included).
- Client-standardised pattern files and client-generated rule files integrated into the commands; the workflow demonstrably runs against the newly covered repositories.

14. Payment Terms & Next Steps

14.1 Payment

Phase 1 is milestone-based against the fixed price of USD 11,000, which includes the IBA microservice reference build (~300,000 LOC) and 40 hours of optimisation & support over the quarter. Phase 2 is quoted and scheduled in an addendum after the audit, billed per repository on delivery. The optional continued-improvement support package, if taken, is billed per quarter (USD 3,500; additional hours at USD 30/hr). Invoices are net 15.

Milestone	Trigger	Amount (USD)
1. Mobilisation	On signing — Phase 1 begins	$5,500 (50% of Phase 1)
2. Phase 1 acceptance	Reference model, audit & training delivered	$5,500 (50% of Phase 1)
3. Phase 2 (addendum)	Per-repo on delivery, per chosen option	Per audit estimate
Phase 1 total		**$11,000**

14.2 Next steps

1. Confirm the IBA microservice as the Phase 1 reference repository (and any further repo) and share current BE/FE coding standards.
2. Provision repository, Confluence, Jira, Figma, Slack and Sentry access and Anthropic capacity (MySQL access if the optional MCP is used).
3. Choose the intended Phase 2 delivery model (Brilworks-led or hybrid) so the audit estimate is framed correctly.
4. Countersign this SOW; Brilworks schedules kickoff within five business days.

Prepared by

Brilworks Technology Pvt Ltd

Vikas Singh — CTO & Co-founder

vikas@brilworks.com • +91-8866518364 • www.brilworks.com

503, Fortune Business Hub, Science City Road, Sola, Ahmedabad, Gujarat, India 380060